Reporte de tutoría guiada

Palíndromo recursivo en Java con patrón wrapper

Alumno: Said Miguel Meza Lopez · Grado: 4A
Materia: Estructura de Datos · Fecha: 14 de octubre de 2026

Said #1

Hola, mi nombre es Said Miguel Meza Lopez, del grado 4A. Estoy en la materia de Estructura de Datos y necesito que actúes como mi tutor guiado. Quiero diseñar un método recursivo en Java para verificar si una palabra es palíndromo usando el patrón wrapper. No me des el código resuelto; hazme preguntas para guiarme en el análisis del caso base, el paso recursivo y la implementación.

Claude · tutor #2

¡Hola, Said! Con gusto te guío. El código lo vas a escribir tú. Empecemos entendiendo el problema antes de pensar en Java.

Con la palabra **"radar"**, sin código: ¿qué dos letras compararías primero? Si coinciden, ¿qué te queda por revisar y cómo describirías ese "resto"? Con **"casa"**, ¿en qué momento te darías cuenta de que no es palíndromo y qué harías?

Pista para el caso base: ¿qué situaciones harían que ya no tengas nada más que comparar? Prueba con una palabra par ("anna") y una impar ("radar").

Said #3

Compararía los extremos. Si coinciden, me queda lo que está dentro de los extremos, "lo de en medio". Me daría cuenta de que no es palíndromo cuando los extremos no sean iguales.

Claude · tutor #4

¡Muy bien! Sin darte cuenta describiste la estructura recursiva: el "resto" es **el mismo problema, pero más pequeño**.

Ahora afinemos: ¿qué debe devolver el método cuando los extremos no coinciden y ya puedes parar? Si usamos `izq` y `der` como posiciones, ¿cómo cambian en cada paso? Sigue "anna" (0,3) y "radar" (0,4) y busca qué tienen en común `izq` y `der` al final, para formular el caso base.

— FASE 2 · CASO BASE Y PASO RECURSIVO —

Said #5

Puede devolver un false, o también un mensaje diciendo que no es palíndromo. Ya no se necesita revisar el resto, se podría parar. Lo de en medio de "radar" son las posiciones 1, 2, 3. `izq` y `der` van aumentando su posición. Lo que tienen en común: izq = der.

Claude · tutor #6

Hay aciertos y cosas por afinar:

- Correcto: cuando los extremos difieren se puede parar, y lo de en medio de "radar" es de la posición 1 a la 3.
- Revisa "radar": (0,4) → (1,3). ¿`der` aumentó? Corrígelo con tus palabras.
- La condición `izq = der` no se cumple en "anna". Busca una que cubra longitudes pares e impares.
- Sobre devolver un `String`: en recursión, el resultado de una llamada se usa dentro de la anterior. ¿Qué tipo de dato simple sirve para un `if`?

Adelanto del wrapper: ¿con qué valores empieza siempre la primera llamada? ¿Debería escribirlos el usuario?

Said #7

Opino que un `String` sería lo mejor, solo para que mande a llamar el método. La izquierda aumenta y la derecha disminuye una posición. La condición es `izq >= der`. La primera llamada empieza con `izq` en 0 y `der` con longitud − 1. El usuario no debería escribirlos: solo escribe la palabra y el algoritmo se encarga.

Claude · tutor #8

¡Muy bien! Correcto: `izq` aumenta, `der` disminuye, `izq >= der` cubre pares e impares y el usuario no escribe esos valores (ahí entra el wrapper).

Falta afinar el tipo de retorno: si la llamada para "lo de en medio" devolviera `"Es palíndromo"`, ¿tendrías que comparar textos dentro del `if`? Tu intuición de `false` era buena: ¿qué tipo de dato en Java tiene solo dos valores? El mensaje bonito lo imprime quien llama al método.

Ahora diseña la estructura: parámetros y retorno del recursivo, qué devolver en el caso base, qué devolver si los extremos son distintos o iguales, y qué recibe el wrapper.

— FASE 3 · DISEÑO DE LOS DOS MÉTODOS —

Said #9

El método recursivo devuelve un booleano. En el caso base devuelve `true`, porque la palabra es igual. Si los extremos son distintos devuelvo `false`. Si son iguales continúa aumentando y disminuyendo el método. El wrapper pasa 0 y palabra.length − 1.

Claude · tutor #10

Ya tienes casi todo el diseño. Correcto: `boolean`, caso base `true`, `false` si los extremos difieren, y el wrapper arranca con `0` y `palabra.length() - 1` (con paréntesis).

Falta: escribir la firma del recursivo con sus tres parámetros, concretar qué valores exactos le pasas a la nueva llamada, decidir si el resultado se guarda o se devuelve directo, y decir qué recibe y devuelve el wrapper. Arma el esqueleto de los dos métodos.

Said #11

Método recursivo:

```
private static boolean esPalindromoRec( String palabra , Int izq , Int der)
```

Después `izq + 1` y `der - 1`; el resultado lo puedo devolver directamente con return.

```
public static boolean esPalindromo( palabra) {
    // retornaría el método, pasando 0 y palabra.length-1
}

private static boolean esPalindromoRec(string palabra, int izq, int der) {
    if(izq>= der) true
    else false
    true
}
```

Claude · tutor #12

Vas muy bien. Detalles de sintaxis: `String` y `int` se escriben así (no `string` ni `Int`), el wrapper necesita el tipo del parámetro, `length()` lleva paréntesis y faltan los `return`.

Lo importante está en el cuerpo: tu `if/else` mezcla dos cosas. Haz la prueba de escritorio con "radar" (0,4): ¿se cumple `izq >= der`? Separa los **tres casos**: nada por comparar, extremos distintos, extremos iguales (delegar). Te dejé un esqueleto con espacios para completar.

Said #13

```
private static boolean esPalindromoRec(String palabra, int izq, int der) {
    if (izq>=der) {
        return true;      // caso base
```

```
    }
    if (palabra.charAt(izq) != palabra.charAt(der)) {
        return false;      // extremos distintos
    }
    return esPalindromoRec(palabra, izq+1, der-1);   // extremos iguales
}
```

Claude · tutor #14

¡Excelente! Tu método recursivo está **correcto**: los tres casos están separados, los `return` bien colocados y la llamada siempre se acerca al caso base.

Te dejé una prueba de escritorio con "casa" y "anna". Falta el wrapper (¿qué tipo es `palabra` y qué tres valores pasas?). Preguntas de reflexión: ¿por qué el recursivo es privado (piensa en `esPalindromoRec("radar", 3, 1)`)? ¿Qué pasa con `""` y con `"a"`?

— FASE 4 · EL WRAPPER —

Said #15

```
public static boolean esPalindromo(Stringpalabra) {
    return esPalindromoRec(palabra, 0, palabra-length-1);
}
```

Claude · tutor #16

Casi: la estructura está bien (devuelve lo del recursivo y pasa `palabra`, `0` y el último índice). Dos detalles: falta el espacio en `Stringpalabra`, y `palabra-length-1` Java lo lee como una resta; `length()` es un método y se llama con punto y paréntesis.

Pendientes: la prueba de "casa", por qué el recursivo es privado y los casos límite.

Said #17

Prueba con "casa": se devuelve `false` en la primera llamada porque los extremos no son iguales; no hay segunda llamada. Si el recursivo no fuera privado, `esPalindromoRec("radar", 3, 1)` devolvería `true`, y `private` limita el acceso desde fuera de la clase. Con `""` la llamada es `"", 0, -1`; con `""` y `"a"` se cumple el caso base.

```
public static boolean esPalindromo(String palabra) {
    return esPalindromoRec(palabra, 0, palabra-length(String)-1);
}
```

Claude · tutor #18

Tus respuestas de análisis son correctas. Ojo con "radar", 3, 1: "radar" sí es palíndromo, pero el método contestó sin comparar nada; con "casa" también daría `true` y sería falso. Por eso el recursivo es privado. Con `""` no hay error porque nunca se llama a `charAt`.

El wrapper aún tiene un error: `palabra-length(...)` usa un guion en vez de punto, y `length()` no necesita argumentos. Piensa en cómo escribiste `palabra.charAt(izq)`.

Said #19

```
public static boolean esPalindromo(String palabra) {
    return esPalindromoRec(
        palabra, 0, palabra.length() - 1
    );
```

Claude · tutor #20

¡Muy bien! El tercer argumento ya está correcto. Solo falta la llave de cierre `}` del método.

Siguientes pasos: arma un `main` con "radar", "anna", "casa", `""` y "a" anotando lo esperado; reto de "Radar" (¿qué método de `String` pasa todo a minúsculas?); y reflexiona por qué la solución es recursiva y no iterativa.

— FASE 5 · EL MAIN Y LAS PRUEBAS —

Said #21

```
public static void
```

Claude · tutor #22

Tu mensaje se cortó en `public static void`. La firma del `main` es `public static void ___(String[] args)`: ¿cómo se llama el método con el que Java inicia? Dentro, ¿cómo imprimirías el `boolean`? Anota antes lo esperado por cada palabra. Recuerda que todo va dentro de una clase.

Said #23

```
public static void ___(String[] args) {
    String[] palabras= {"radar", "anna", "","a"}
    if(esPalindromo(palabra){System.out.println( palabra + "Es
palíndromo"); else {System.out.println( + palabra +" No es palíndromo");}}}
```

Claude · tutor #24

Buen intento: ya usas un arreglo y un `if/else`. Por corregir: el nombre `main`, el `;` del arreglo, que `palabra` no existe (necesitas un ciclo `for (String palabra : palabras)`), los paréntesis y llaves del `if/else`, el `+` sobrante y los espacios del mensaje. Además faltó "casa", el caso que debe dar `false`.

Said #25

```
public static void main(String[] args) {
    String[] palabras = {"radar", "anna", "casa", "", "a"};
    for (String palabra : palabras) {
        if (esPalindromo(palabra)) {
            System.out.println(palabra + "Es palíndromo");
        } else {
            System.out.println(palabra + "No es palíndromo");
        }
    }
}
```

Claude · tutor #26

¡Muy bien! El `main` está bien estructurado y compilaría. Detalle: `palabra + "Es palíndromo"` imprimiría `radarEs palíndromo`, y con `""` no se distingue qué se evalúa. Usa comillas escapadas (`"\""`) y un separador.

Pendientes: comentarios con lo esperado, la clase, el reto de "Radar" y la reflexión final.

Said #27

```
public static void main(String[] args) {

    /*
     * Pruebas esperadas:
     * radar -> Es palíndromo
     * anna  -> Es palíndromo
     * casa  -> No es palíndromo
     * ""    -> Es palíndromo
     * a     -> Es palíndromo
     */

    String[] palabras = {"radar", "anna", "casa", "", "a"};

    for (String palabra : palabras) {
        if (esPalindromo(palabra)) {
            System.out.println("\"" + palabra + "\" -> Es palíndromo");
        } else {
            System.out.println("\"" + palabra + "\" -> No es palíndromo");
        }
```

```
    }
}
```

Claude · tutor #28

¡Excelente! Los comentarios con lo esperado están completos y correctos, y el formato de salida quedó mucho mejor.

Te falta armar la clase completa (el nombre del archivo debe coincidir con la clase pública), cerrar la llave del wrapper, resolver el reto de "Radar" con `toLowerCase()` en el wrapper y escribir tu reflexión final.

— FASE 6 · CLASE COMPLETA Y REFLEXIÓN —

Said #29

Clase completa con el wrapper que convierte a minúsculas y pruebas incluyendo "Radar":

```
public class Palindromo {

    // Wrapper que recibe la palabra e inicia la recursividad
    public static boolean esPalindromo(String palabra) {
        palabra = palabra.toLowerCase();
        return esPalindromoRec(
            palabra, 0, palabra.length() - 1
        );
    }

    // Metodo recursivo
    private static boolean esPalindromoRec(
        String palabra, int izq, int der
    ) {
        if (izq >= der) {
            return true;
        }
        if (palabra.charAt(izq) != palabra.charAt(der)) {
            return false;
        }
        return esPalindromoRec(
            palabra, izq + 1, der - 1
        );
    }

    // main con pruebas: radar, anna, casa, "", a, Radar
}
```

Claude · tutor #30

¡Felicidades! Tu clase está **completa y correcta**. El reto de "Radar" quedó resuelto en el wrapper sin tocar el recursivo: esa es una ventaja real del patrón. Se imprime la palabra original porque `toLowerCase()` devuelve un nuevo `String` (los `String` son inmutables).

Pendiente: la reflexión final. ¿Qué parte es "el mismo problema, pero más pequeño"? ¿Por qué la recursión siempre termina? Retos opcionales: espacios (`replace`) y qué pasa con `null`.

Said #31

1. "El mismo problema, más pequeño" es la llamada recursiva, porque el método vuelve a llamarse pero ahora con lo de adentro.

2. La recursión termina porque en cada llamada `izq` aumenta una posición y `der` disminuye una, y se detiene cuando `izq >= der`.

Reto espacios: `palabra = palabra.replace(" ", "").toLowerCase();`

Reto `null`: falla con `null` y se resuelve con `if (palabra == null) { return false; }`

Claude · tutor #32

¡Muy bien! Reflexión **correcta**. Para redondearla: cada llamada reduce la distancia `der - izq` en 2; como es un entero que siempre baja, tarde o temprano se cumple `izq >= der`.

Sobre `null`: `toLowerCase()` fallaría con `NullPointerException`, y tu `if` evita el error, pero debe ir **antes** de `toLowerCase()`. Devolver `false` es una decisión de diseño; lanzar una excepción también sería válido.

- **Caso base:** `izq >= der`, cubre longitudes pares e impares.
- **Paso recursivo:** comparar extremos y delegar el interior con `izq + 1` y `der - 1`.
- **Wrapper:** oculta los parámetros técnicos y prepara los datos sin tocar la recursión.
- **Terminación:** cada llamada acerca `izq` y `der`, así el caso base siempre se alcanza.

— FASE 7 · CÓDIGO FINAL —

Said #33

Mi clase final con todas las mejoras:

```
public class Palindromo {

    // Wrapper que prepara la palabra e inicia la recursividad
    public static boolean esPalindromo(String palabra) {

        // Validar que la palabra no sea null
        if (palabra == null) {
            return false;
        }
```

```
        // Convierte a minusculas y eliminar espacios
        palabra = palabra.replace(" ", "").toLowerCase();

        // Iniciar la recursividad desde los extremos
        return esPalindromoRec(
                palabra, 0, palabra.length() - 1
        );
    }

    // Metodo recursivo
    private static boolean esPalindromoRec(
            String palabra, int izq, int der
    ) {

        // Caso base: llegamos al centro
        if (izq >= der) {
            return true;
        }

        // Si los extremos son diferentes no es palindromo
        if (palabra.charAt(izq) != palabra.charAt(der)) {
            return false;
        }

        // Continuar con la parte interior de la palabra
        return esPalindromoRec(palabra, izq + 1, der - 1);
    }

    // Metodo principal para probar el programa
    public static void main(String[] args) {

        /*
         * Resultados esperados:
         * radar -> Es palindromo
         * anna -> Es palindromo
         * casa -> No es palindromo
         * "" -> Es palindromo
         * a -> Es palindromo
         * Radar -> Es palindromo
         * anita lava la tina -> Es palindromo
         */

        String[] palabras = {"radar", "anna", "casa", "", "a",
                             "Radar", "anita lava la tina"};

        for (String palabra : palabras) {
            if (esPalindromo(palabra)) {
                System.out.println("\"" + palabra + "\"  Es palindromo");
            } else {
                System.out.println("\"" + palabra + "\"  No es
palindromo");
            }
        }
    }
}
```

Claude · tutor #34

¡Ya está, Said! Tu código final es correcto: el `null` va primero, luego `replace` y `toLowerCase()`, y al final el `return` que arranca la recursión. Los tres casos del recursivo están bien y todas las llaves cierran.

Fin. Buen trabajo, y suerte con la entrega. 🎉